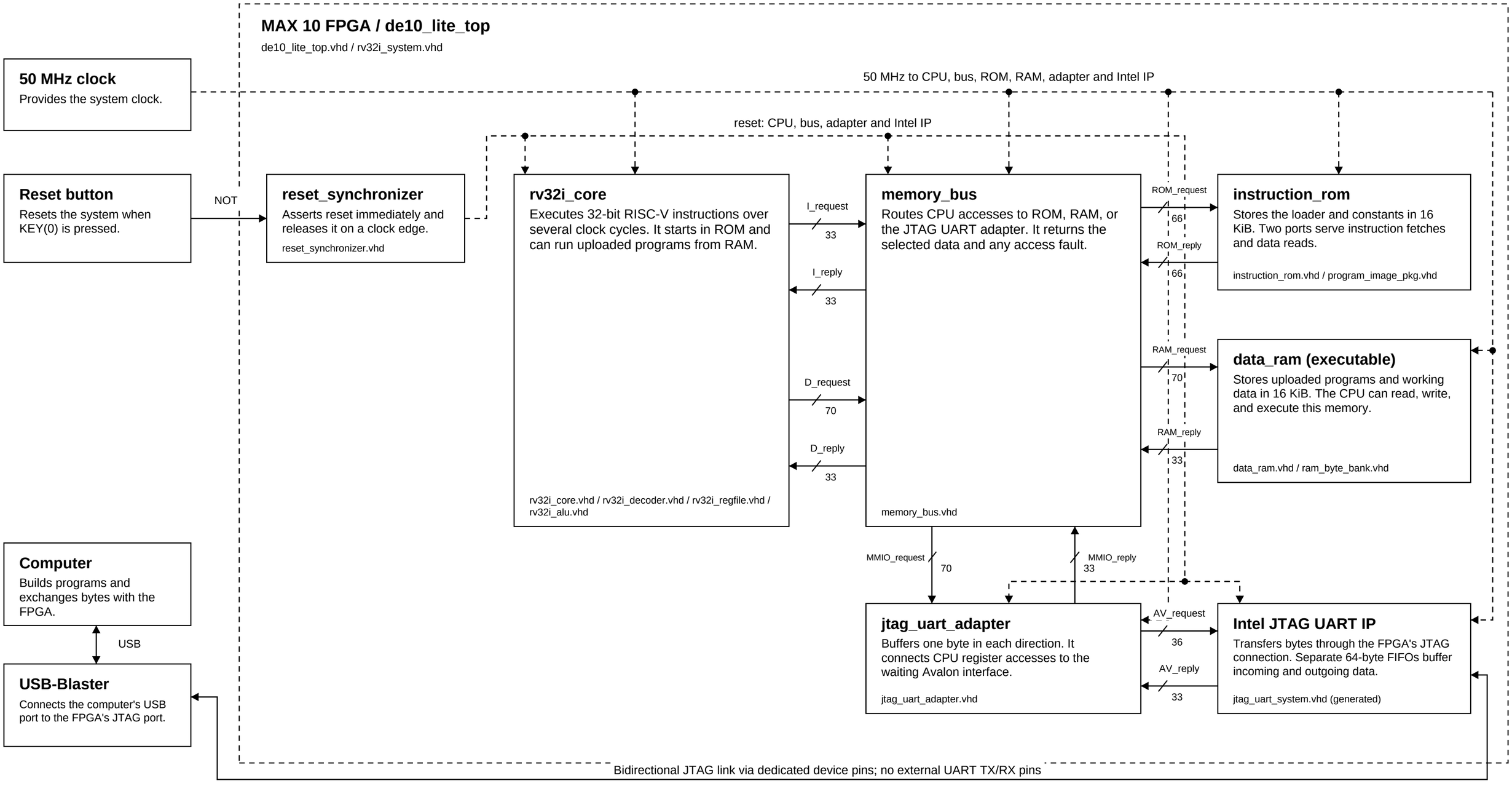
RV32I System Diagram - Top Level Diagram



ARROW KEY

I = instruction; D = data; request = command to a block; reply = result. Numbers in parentheses are bits; widths add the separate signals.

CPU arrows

I_request (33): address (32) + request (1).
I_reply (33): instruction (32) + fault (1).
D_request (70): address (32) + write data (32) + byte enables (4) + request (1) + write (1).
D_reply (33): read data (32) + fault (1).

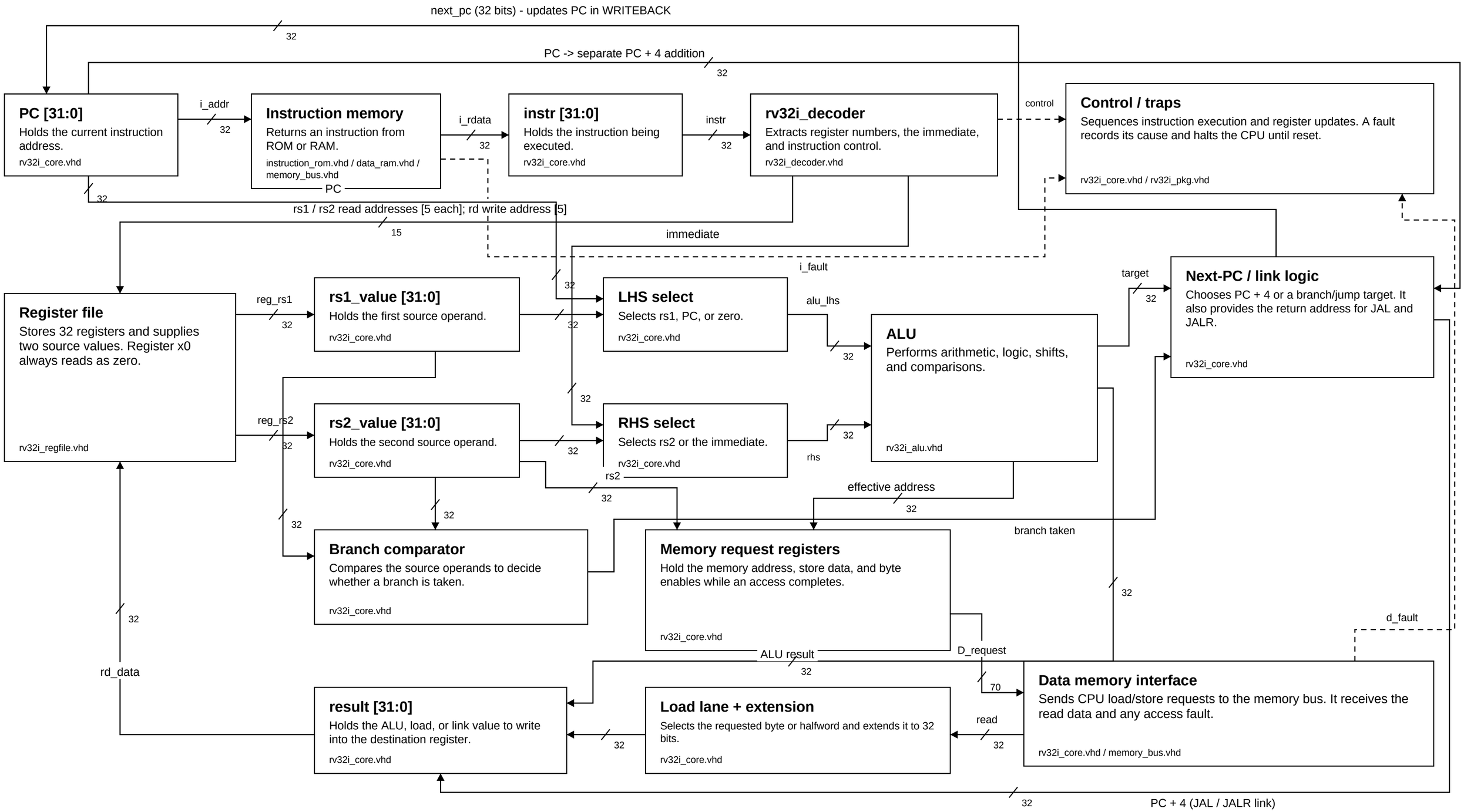
Memory and MMIO arrows

ROM_request (66): two addresses (32 each) + two requests (1 each).
ROM_reply (66): two words (32 each) + two faults (1 each).
RAM_request / MMIO_request (70): same fields as D_request.
RAM_reply / MMIO_reply (33): read data (32) + fault (1).
Byte enables select which of the four bytes to write.

Avalon and external connections

AV_request (36): write data (32) + register select (1) + chip select (1) + read (1) + write (1).
AV_reply (33): read data (32) + wait (1).
Clock and reset: one bit each; dashed arrows.
USB / JTAG: byte traffic carried by the external protocols.

RV32I System Diagram - CPU Datapath and Control



ARROW KEY

I = instruction; D = data; request = command to a block; reply = result. Numbers in parentheses are bits; widths add the separate signals.

Instruction, register, and address arrows

PC / i_addr / next_pc / PC + 4: one 32-bit address each.
i_rdata / instr: one 32-bit instruction each.
rs1 / rs2 / rd: three 5-bit register numbers (15 total).
reg_rs1 / reg_rs2 / rs1_value / rs2_value:
one 32-bit register value on each arrow.

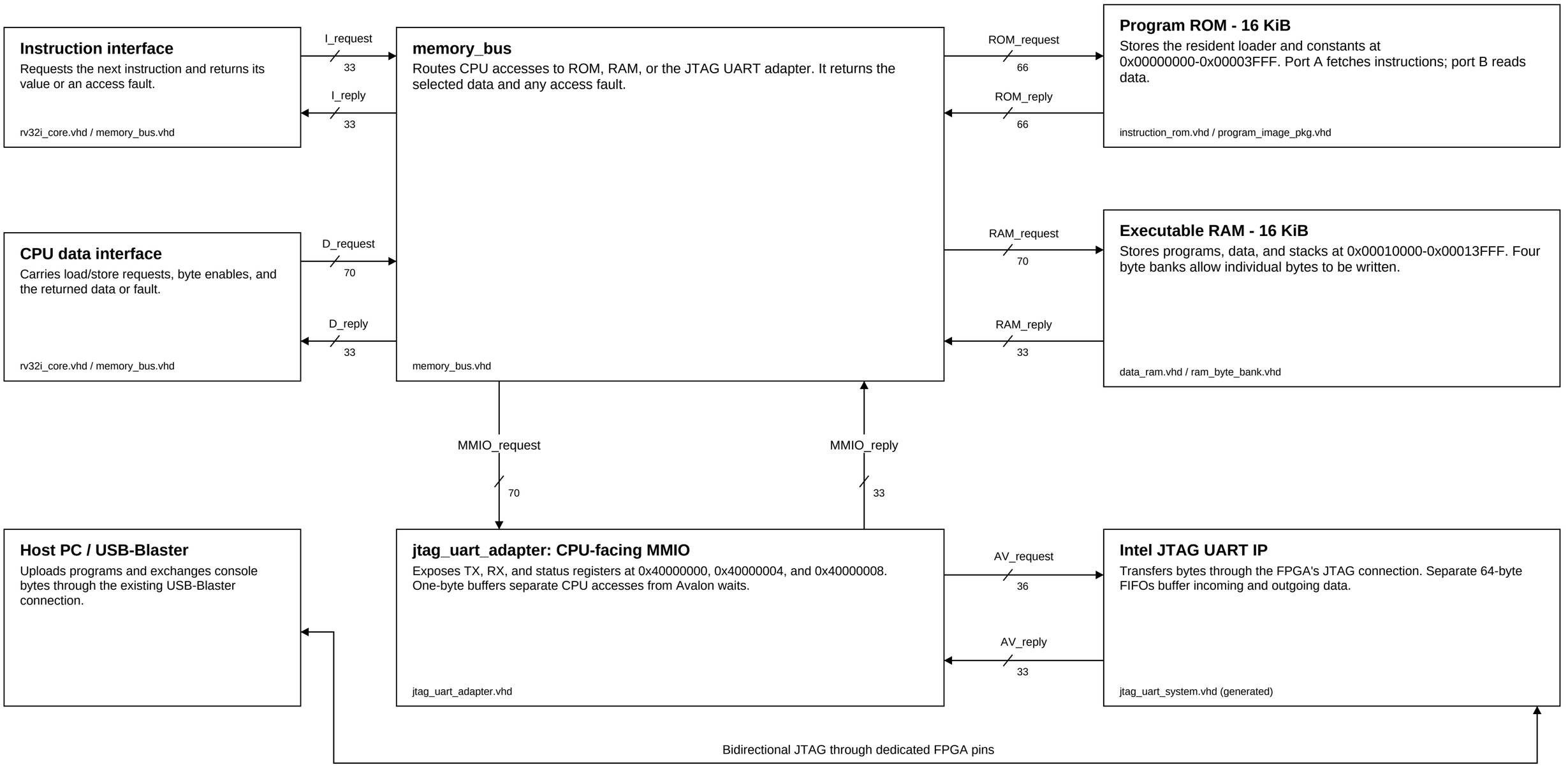
ALU and writeback arrows

immediate: one 32-bit instruction constant.
alu_lhs / rhs: one 32-bit ALU input each.
target / effective address / ALU result: 32 bits each.
rs2 / read / rd_data: one 32-bit data value each.
branch taken / i_fault / d_fault: one decision bit each.

Memory and control arrows

D_request (70): address (32) + write data (32)
+ byte enables (4) + request (1) + write (1).
control: instruction kind, ALU operation, operand choices, writeback choice, branch condition, and load/store options.
The control fields are a VHDL record, not a fixed-width bus.

RV32I System Diagram - Memory and JTAG UART Routing



ARROW KEY

I = instruction; D = data; request = command to a block; reply = result. Numbers in parentheses are bits; widths add the separate signals.

CPU and ROM arrows

I_request (33): address (32) + request (1).
I_reply / D_reply (33): word (32) + fault (1).
ROM_request (66): two addresses (32 each) + two requests (1 each).
ROM_reply (66): two words (32 each) + two faults (1 each).

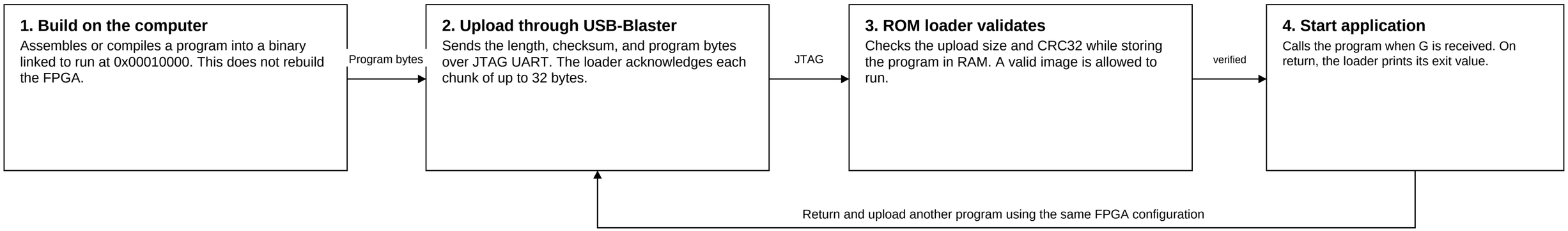
Data, RAM, and MMIO arrows

D_request / RAM_request / MMIO_request (70):
address (32) + write data (32) + byte enables (4)
+ request (1) + write (1).
RAM_reply / MMIO_reply (33): read data (32) + fault (1).
Each byte-enable bit selects one of the four word bytes.

Avalon and JTAG arrows

AV_request (36): write data (32) + register select (1)
+ chip select (1) + read (1) + write (1).
AV_reply (33): read data (32) + wait (1).
Data register: byte [7:0], valid [15]; control: TX space [31:16].
JTAG link: incoming and outgoing bytes through USB-Blaster.

RV32I System Diagram - Program Upload and Execution



RAM layout and ownership

Divides RAM into space for the application, its stack, and the loader. These regions are a software convention.
Application image / data: 0x00010000-0x00012FFF (12 KiB)
Application stack: 0x00013000-0x00013BFF (3 KiB)
Loader workspace / stack: 0x00013C00-0x00013FFF (1 KiB)

Protocol and example session

Uses L to upload, G to run, and ? for help. A successful upload reports LOADED before G can start the program.
Example: upload -> LOADED -> G -> RUN
Program output -> EXIT 0000002A -> ready>

ARROW KEY

Arrows show the order of the upload/run process and the bytes exchanged.

Program bytes / JTAG arrows

Program bytes: the compiled instructions and initial data.
Upload header: L (1 byte) + length (4 bytes) + CRC32 (4 bytes).
The image follows in chunks of up to 32 bytes.
Multi-byte header values send their lowest byte first.

Verified / return arrows

Verified: size and checksum passed; G may start the program.
Return path: program output, exit value, and the next prompt.
Each + reply permits the next upload chunk.
These are software-flow arrows, not parallel hardware buses.

Values carried in the byte stream

Each transmitted byte is 8 bits.
Length and CRC32 are 32-bit values, sent as four bytes each.
The exit value is 32 bits, printed as eight hex characters.
CRC32 checks the received bytes for transfer corruption.